Real Estate AVM: 7-minute video script

Nov 13, 2026 · @Sherlyn

How to use this script

The video runs about 7 minutes: about 1,100 spoken words at a normal pace (about 160 words a minute), plus a few seconds for clicks. Manasa and Sherlyn each speak about half of it. If a practice run goes over 7:30, skip the Other areas line in the demo.

Part	Time	Speaker	Screen
Introduction	0:00 to 0:40	Manasa	Website home page
Problem and question	0:40 to 1:30	Sherlyn	Home page, then a property website (any)
Data and pipeline	1:30 to 2:30	Sherlyn	Code editor: data_processing folder
Model and results	2:30 to 3:30	Manasa	Results table in the viva guide or reports folder
Website demo	3:30 to 6:00	Both	The live website
Tech stack	6:00 to 6:40	Manasa	Architecture picture in this doc, then the code folders
Limits and closing	6:40 to 7:00	Sherlyn	Website home page

Before recording:

1. Start the app with `bash start_app.sh` and open http://localhost:5173.
2. Hard refresh the page (Cmd+Shift+R) so it shows the latest version.
3. Do one practice check so the server is warm.
4. Close other tabs and turn off notifications.
5. Record the screen with QuickTime (File, New Screen Recording) or OBS, with the microphone on.

Tips: speak slowly and smile. Point with the mouse at what you are talking about. If you make a mistake, pause for 2 seconds and repeat the sentence; you can cut the gap later. Lines in quotes are what you say; lines in brackets are what you do on screen.

0:00 to 0:40 · Introduction (Manasa)

[Screen: the website's Check price page, sidebar visible.]

Manasa: "Hello everyone. I am Manasa, and this is Sherlyn. This is our OJT project, Real Estate AVM. AVM stands for Automated Valuation Model: software that estimates what a property is worth using data instead of a human valuer.

We built it as a fair-price checker for flats in Bangalore. You find a flat on any property website, you give us its details, and in a few seconds we tell you a fair price at today's market level, whether the seller's price is too high, fair or low, and why.

In this video we will cover the problem, our data, the machine learning model, a full demo of the website, and the technology behind it."

0:40 to 1:30 · The problem and research question (Sherlyn)

[Screen: stay on the home page; optionally show any property website for 3 seconds.]

Sherlyn: "Thank you, Manasa. I am Sherlyn. Buying a flat is the biggest purchase most families make, but it is very hard to know if a price is fair. Two flats of the same size can be listed at very different prices, and a manual valuation takes time and money.

So we asked one clear research question: does adding location and nearby amenity data, like schools, hospitals and transport, really improve price prediction over the flat's own details alone, and by how much?

We can measure it by comparing the error with and without location, and as you will see, it has a clear answer."

1:30 to 2:30 · Data and pipeline (Sherlyn)

[Screen: code editor with the data_processing folder open. Click each file as you name it.]

Sherlyn: "Our data is the public Bengaluru House Price dataset from Kaggle: 13,320 real listings with size, bedrooms, bathrooms, location and price.

The raw data was messy. Sizes were written as ranges like 1200 to 1400, or in square metres, and bedrooms were text like '2 BHK'. In load_bangalore.py we cleaned all of this and removed impossible rows, like five bedrooms in 600 square feet.

Then we found where each area is. We geocoded every area name to a latitude and longitude with OpenStreetMap's Nominatim service. Next, compute_amenity_distances.py asked the OpenStreetMap Overpass API for the nearest school, hospital and bus or train stop for every area.

After cleaning and trimming the extreme 1 percent at each end, we had 11,272 listings across 881 areas.

We also built price factors that bring the old 2017 to 2019 prices up to 2026, using published area rates."

2:30 to 3:30 · Model and results (Manasa)

[Screen: the results table (Part 5 of the viva guide, or reports/model_comparison.csv).]

Manasa: "Then I trained and compared models. We split the data 80 to 20: the models learn from 9,017 flats and are tested on 2,255 flats they have never seen.

First, a simple baseline: the area's usual rate per square foot times the size. Then linear models, Random Forest and XGBoost, each trained twice: once with only the flat's own details, and once with the location features added.

The result was very clear. With only size, bedrooms and bathrooms, even XGBoost could not beat the simple baseline. Adding location cut the error by 18 percent, and the R squared rose from 0.60 to 0.73.

After tuning, our final XGBoost model is about 24 percent off on average, with an R squared of 0.75.

We also explain every estimate with SHAP. SHAP shows how much each detail pushed the price up or down, and the parts add up exactly to the prediction. Across all flats, the flat itself explains about 70 percent of the price and location about 30 percent. And the biggest location effect is which area you are in, not how close the nearest school is."

3:30 to 6:00 · Website demo, page by page (both)

Check price (Sherlyn, 3:30 to 4:30)

[Click **Try an example**, then **Fill from ad**.]

Sherlyn: "Let me show you the website. We kept every word simple, so anyone can use it. Say I found this ad online. I paste it here and click Fill from ad. The website reads the area, size, bedrooms, bathrooms and price by itself: a 2 BHK in Whitefield, 1,200 square feet, at 1.75 crore. Under the area it shows the usual price there, about 14,000 rupees per square foot."

[Click **Check price**.]

Sherlyn: "The answer is big and clear: Price is too high. The seller asks 12 percent more than our price of 1.57 crore. This bar shows the fair range, 1.41 to 1.72 crore, and the red pin is the seller's price, just outside it. The tips tell you what to do: offer about 1.57 crore, and

do not pay more than 1.94 crore."

[Open **Why this price?**]

Sherlyn: "If you want the reason, open Why this price. Green adds to the price and red lowers it. This is the SHAP explanation Manasa talked about. At the end, prices in Whitefield went up about 2.5 times since our data was collected, which gives today's price."

Smart checks (Manasa, 4:30 to 5:00)

[Type **20** in *How old is the building?* and click Check price.]

Manasa: "Our data has no building age, so we added the rule that property valuers use: a building lasts about 60 years, and only the building wears out, not the land. For a 20-year-old building the fair price becomes about 1.48 crore, and the tips tell you to check for leaks, the lift and the pipes."

[Change the seller's price to **3** with **Lakhs** selected and click Check price.]

Manasa: "People often mix up lakhs and crores. If someone types 3 lakhs by mistake, we do not call it a bargain. We ask: did you mean 3 crore? One click fixes it."

The other pages (both, 5:00 to 6:00)

[Click **Compare** in the sidebar.]

Manasa: "Compare shows the price per square foot of similar flats. Our price and the seller's price are marked, and the seller's price is costlier than about 71 percent of similar flats. These are old records, not flats for sale, and we say that clearly."

[Click **Places nearby**. Type a society name and click **Find**.]

Sherlyn: "Places nearby lists the 5 closest schools, hospitals and bus or metro stops, with a map. If you type the society name, we measure from that exact spot."

[Click **Other areas**.]

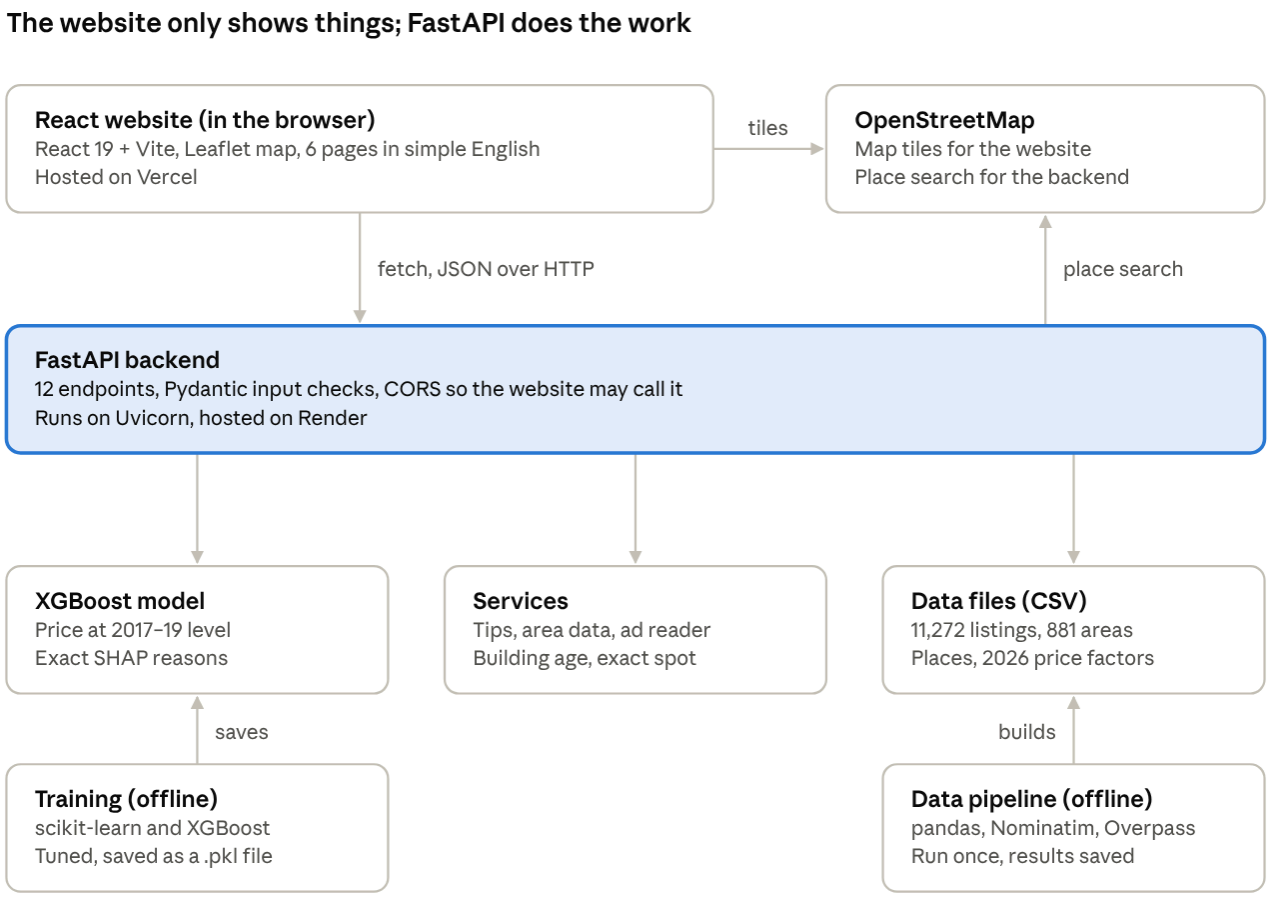
Sherlyn: "Other areas shows nearby areas that are cheaper or costlier, in case Whitefield is over budget."

[Click **Before you buy**, then **Loan EMI**; type a salary.]

Manasa: "Before you buy is a simple checklist: visit the flat, check the RERA number and the papers, and never pay without a written agreement. And Loan EMI shows the monthly payment. It checks the bank's down-payment rule, and if you add your salary, it suggests the best loan length for you."

6:00 to 6:40 · Tech stack and architecture (Manasa)

[Screen: show this picture full screen, then the backend and frontend folders in the code editor.]



architecture · website, API, model, data

Manasa: "Here is how it is built. Everything is in Python, except the website, which is JavaScript.

The website is made with React and Vite, and the map uses Leaflet with OpenStreetMap. It only shows things. All the work happens in our FastAPI backend, which has 12 endpoints. It runs the XGBoost model and the SHAP explanation, and small Python services give the tips, read the pasted ad, find nearby places and apply the building-age rule.

The data work used pandas and OpenStreetMap, and the model was trained with scikit-learn and XGBoost. We tested everything with 73 automated tests in pytest, used git and GitHub, and deploy the backend on Render and the website on Vercel."

6:40 to 7:00 · Limits and closing (Sherlyn)

[Screen: back to the Check price page.]

Sherlyn: "We want to be honest about the limits. Our data has asking prices, not final sale prices, and locations are known only at area level. So our estimate is a strong first check, not a bank valuation.

To sum up: location really matters, about 30 percent of the price, and our website turns that into a simple, explained fair-price check that anyone can use. Thank you for watching."

Manasa: "Thank you."

[Both smile. Stop recording after 2 seconds.]